ペンプロッターにおけるストローク描画順の最適化

電気通信大学 II 類
2412562 川島聡真

提出日：2026 年 10 月 25 日

1 はじめに

1.1 問題設定

ペンプロッターで絵や文字を描くとき，画像処理の結果として得られるのは「ベジェ曲線をつないだストローク」の集まりである．各ストロークはペンを下ろしたまま一筆で描く線であり，始点と終点をもつ．ストロークとストロークの間はペンを上げて移動する（以下 空走と呼ぶ）．空走は紙に何も描かないので，短いほど描画時間が短くなる．

ストロークの中身（ベジェ曲線の形）はすでに決まっているので，描く順番を変えても「描く距離」は変わらない．変えられるのは次の 2 つだけである．

1. どのストロークをどの順番で描くか
2. 各ストロークを始点から描くか，終点から逆向きに描くか

つまり問題は「入口と出口が異なる点の巡回セールスマン問題（TSP）」になる．通常の TSP は都市（点）を巡るが，ここでは都市が 長さをもった線分 で，入った端点と反対の端点から出ていく．また，プロッターは描き終えたら原点に戻る必要がないので，閉路ではなく 開路（原点から出発して最後のストロークで終わる）である．

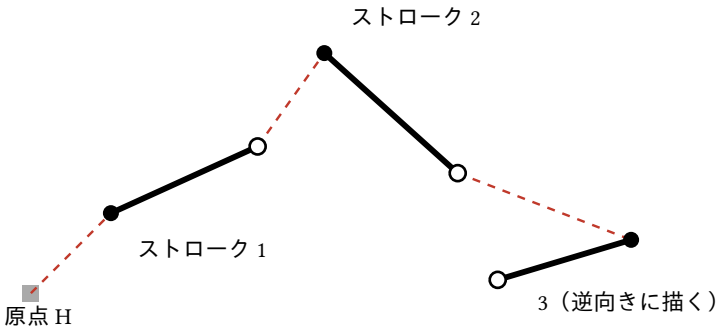


図 1: 描画順の例．黒の太線が描画，赤の破線が空走．●が入口，○が出口．ストローク 3 は終点側から入ると空走が短くなる

1.2 定式化

ストローク v の始点を s_v，終点を e_v とする．描く順番を並び $\pi = (\pi_1, ..., \pi_N)$，向きを $f_v \in \{0, 1\}$（1 なら逆向き）で表す．入口 $\mathrm{in}(v)$ と出口 $\mathrm{out}(v)$ は

$$\mathrm{in}(v) = \begin{cases} s_v (f_v = 0) \\ e_v (f_v = 1) \end{cases}, \quad \mathrm{out}(v) = \begin{cases} e_v (f_v = 0) \\ s_v (f_v = 1) \end{cases} \tag{1}$$

であり，最小化したい空走距離は

$$L(\pi, f) = d(H, \mathrm{in}(\pi_1)) + \sum_{k=1}^{N-1} d\big(\mathrm{out}(\pi_k), \mathrm{in}(\pi_{k+1})\big) \tag{2}$$

となる．ここで H は原点，d は 2 点間の距離である．

TSP は NP 困難であり，ストローク数 N が数千〜数万になると最適解を求めるのは現実的でない．しかし本用途では 最適解は不要 で，「どんな端末（スマートフォンを含む）でも数百ミリ秒以内に，十分よい順番が出ること」が重要である．そこで本レポートでは 構築法で素早く初期解を作り，局所探索で時間の許す限り改善する という，実用 TSP ソルバの定石に沿った構成を検討する．

2 全体の方針

アルゴリズムは役割で 2 種類に分けられる．

- 構築法：何もない状態から 1 つの順番を作る．速いが質はそこそこ．（最近傍法，空間充填曲線，貪欲辺法）
- 改善法 :既にある順番を少しずつ組み替えて短くする．時間をかけるほど良くなり，途中で打ち切ってもその時点の解が使える．（2-opt，Or-opt など）

ランダムな点の TSP について，各手法が最適解（の下界）より何 % 長い解を出すかは [1] によく調べられており，おおよそ 表 1 のとおりである．

表 1: 各手法の解の質の目安（一様乱数点の TSP，[1–3] による概数）

手法	最適からの超過	計算量（工夫後）
空間充填曲線	約 25 %	$O(N \log N)$
最近傍法	約 25 %	$O(N)$〜$O(N \log N)$
貪欲辺法	約 15〜20 %	$O(N \log N)$
2-opt（近傍リスト）	約 5 %	実測でほぼ $O(N)$〜
2-opt + Or-opt	約 4〜5 %	同上
Lin–Kernighan (LKH)	約 2 % 以下	重い・実装が複雑

本問題は端点が 2 つあるため数値はそのままは当てはまらないが，傾向は同じである．以下，各アルゴリズムを順に解説する．

3 アルゴリズム別の解説

3.1 最近傍法（Nearest Neighbor）

3.1.1 考え方

「今ペンがある位置から一番近いストロークへ行く」を繰り返すだけの方法である．ストロークの 2 つの端点のうち近い方を入口に選べば，向きも同時に決まる．

1. ペン位置を原点 H にする．

2. 未描画のストロークの全端点のうち，ペン位置に最も近い端点 p を探す．
3. p を入口としてそのストロークを描く（p が終点なら逆向き）．ペン位置を反対側の端点に移す．
4. 全ストロークを描くまで 2〜3 を繰り返す．

3.1.2 長所と短所

実装が簡単で，見た目にもそれなりに良い順番が出る．一方で「目先の近さ」だけで選ぶので，最後のほうに取り残されたストロークへ遠くまで取りに行く長い空走が発生しやすい．これが約 25 % の超過の主な原因である．

3.1.3 高速化：空間グリッド

素朴に実装すると，毎回すべての未描画ストロークを調べるので $O(N^2)$ になる．現在の `opencv/index.html` の `nearestNeighborOrder` もこの形で，さらに `splice` で配列から要素を抜いているため，N が 1 万を超えると秒単位で遅くなる．

解決策は 空間グリッド である．描画領域を正方形のセルに区切り，各セルにその中の端点を登録しておく．最近傍を探すときはペン位置のセルから外側へリング状にセルを調べていき，見つかった点までの距離が「まだ調べていないリングまでの距離」以下になった時点で打ち切る．端点が平均 1〜2 個入る大きさのセルにすれば，1 回の探索はほぼ定数時間になり，全体で $O(N)$ に近づく．描画済みの端点は削除せずにフラグで飛ばす（遅延削除）と実装が楽である．

3.2 空間充填曲線（ヒルベルト曲線）

3.2.1 考え方

ヒルベルト曲線は，正方形の中のすべての点を 1 本の折れ線で「近いものは近い順に」なぞる曲線である．各ストロークの中点が曲線上のどの位置にあるか（ヒルベルト番号）を計算し，その番号でソートするだけで順番が決まる [3]．向きは，並べた後で前のストロークの出口に近いほうの端点を入口にすればよい．

3.2.2 長所と短所

ソート 1 回なので非常に速く（$O(N \log N)$），実装もビット演算数十行で済む．ただし質は最近傍法と同程度か少し劣る．数十万ストロークの超大規模データ や，低性能端末で即座にプレビューを出したい場合 の保険として持っておくと良い．

3.3 貪欲辺法（Greedy Edge）

3.3.1 考え方

この問題には便利な見方がある．各ストロークは「始点と終点をつなぐ，必ず使う辺」とみなせる．すると順番を決めることは「ある端点の出口と別の端点の入口を空走辺でつなぐペアの選び方」になる．貪欲辺法は，

1. 各端点の近くにある端点との組（候補辺）を作り，短い順に並べる．
2. 短い候補辺から順に，「両端点がまだ空走辺を持っていない」かつ「つなぐと輪ができない」なら採用する．
3. 最後に残った断片（パス）同士を，端点が近い順につなぐ．

という手順で 1 本の経路を組み立てる．輪ができるかの判定には Union-Find（素集合データ構造）を使う．

3.3.2 長所と短所

最近傍法より質が良く（15〜20 % 程度），その後の 2-opt も効きやすい初期解になる．一方で Union-Find や候補辺のソートなど実装量が増える．最初は最近傍法で作り，品質を上げたくなったら貪欲辺法に差し替える という順で進めるのが現実的である．

3.4 2-opt

3.4.1 考え方

2-opt は改善法の基本で，「経路の一部分を丸ごと逆順にする」操作を繰り返す [4]．通常の TSP では，交差している 2 本の辺を組み替えて交差をほどく操作として知られている．

本問題では，位置 i から j までのストロークを逆順に並べ替え，さらにそれぞれのストロークの向きも反転する のがポイントである．こうすると区間の内側の空走はすべて「逆にたどるだけ」になり長さが変わらない．変わるのは区間の両端の 2 本の空走だけである（図 2）．

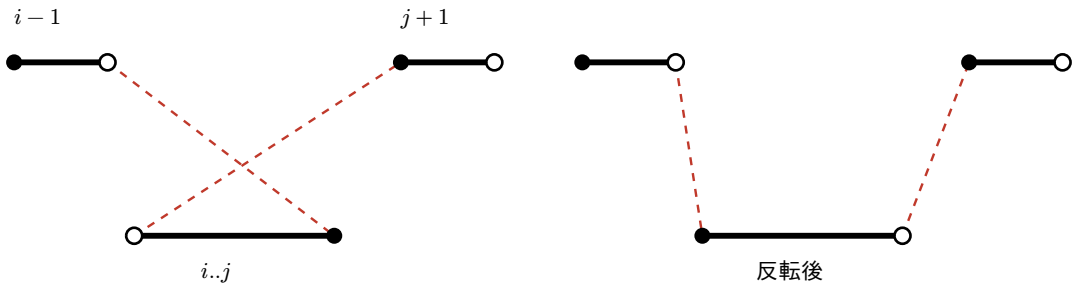


図 2: 2-opt の例（左：改善前，右：区間 $i..j$ を反転した後）．交差していた 2 本の空走がほどけて短くなる．区間が 1 本だけなら「向きの反転」そのものになる

位置 k の入口を E_k，出口を X_k と書くと，区間 $i..j$ を反転したときの空走の減少量（ゲイン）は

$$\Delta = d(X_{i-1}, E_i) + d\big(X_j, E_{j+1}\big) - d\big(X_{i-1}, X_j\big) - d\big(E_i, E_{j+1}\big) \tag{3}$$

であり，**4** 回の距離計算だけで 評価できる．$\Delta > 0$ なら反転を実行する．開路なので，区間が経路の末尾まで続く場合（$j = N$）は E_{j+1} が存在せず，その項は 0 とする．

3.4.2 高速化：近傍リスト

すべての (i, j) の組を調べると 1 周で $O(N^2)$ かかる．そこで 式 3 の新しい辺 $d\big(X_{i-1}, X_j\big)$ に注目する．これが今の辺 $d(X_{i-1}, E_i)$ より短くなければ改善の見込みは薄い．したがって 各端点について近い順に $\boldsymbol{K}$ 個（**8**〜**10** 個程度）の端点を前もって求めておき（近傍リスト），その中だけを相手候補にする [2]．近傍リストは最近傍法と同じ空間グリッドで作れる．候補を近い順に見ていき，距離が今の辺の長さを超えたら打ち切れば，1 周がほぼ $O(NK)$ で済む．

3.4.3 注意点：区間反転のコスト

改善が見つかったときの区間反転は，配列上で $O(j - i)$ の時間がかかる．N が数万を超えるとここが支配的になる（第 4 章の実測を参照）．対策は次のとおり．

- 後述の 分割（グループやタイルごとに最適化）で 1 回あたりの N を小さく保つ．
- 本格的には「2 段リスト」などの反転が速いデータ構造を使う [1]（実装は難しいので，必要になってからでよい）．

3.5 Or-opt

3.5.1 考え方

Or-opt は「連続する 1〜3 本のストロークを抜き出して，別の場所に差し込む」操作である [5]．2-opt が苦手な「ぽつんと離れた 1 本を，本来あるべき近くに移す」改善が得意で，2-opt と交互に使うと効果が高い．差し込むときに塊の向きを反転するかどうかも同時に試す．

抜き出しによる減少量は $d(X_{s-1}, E_s) + d(X_e, E_{e+1}) - d(X_{s-1}, E_{e+1})$，差し込み先（位置 p と $p+1$ の間）での増加量は $d(X_p, E_s) + d(X_e, E_{p+1}) - d(X_p, E_{p+1})$（反転時は E_s と X_e を入れ替える）で，前者が後者より大きければ実行する．差し込み先の候補も近傍リストから選ぶ．

3.6 さらに強い手法（参考）

3-opt，Lin–Kernighan 法（その実装である LKH [6]），焼きなまし法などを使うと最適に近づくが，実装が大きく複雑になる一方で，2-opt + Or-opt からの改善は数 % にとどまる．ペンプロッターの空走は描画時間全体の一部にすぎないので，**Web** アプリでは **2-opt + Or-opt** で打ち止め にするのが費用対効果のよい選択である．

3.7 分割と制約の扱い

3.7.1 グループ順の制約

現在の実装では，ストロークを `structure → fill → detail → delicate` のグループ順に描く制約がある．この場合は グループごとに最適化し，前のグループの最後の出口を次のグループの原点 $\boldsymbol{H}$ として渡す だけでよい．問題が小さく分かれるので，速度面でも有利になる．

3.7.2 大規模データの分割

1 グループが数万本を超える場合は，用紙を数個のタイルに分け，タイルの巡回順（蛇行順など）を決めてから，各タイルの中を最適化する．タイル境界で少し損をするが，区間反転のコストが小さくなり全体はむしろ速く良くなることが多い．

3.7.3 前処理：つながっているストロークの結合

あるストロークの終点と別のストロークの始点が（許容誤差内で）一致しているなら，最適化の前に 1 本に結合してしまう．ペンの上げ下ろし自体が消えるので，順番の最適化よりも効果が大きい場合がある．

4 実測による比較

上記の最近傍法（グリッド版），2-opt，Or-opt を JavaScript で実装し（付録の `order.js`），Node.js v22 で比較した．データは 300 × 300 の領域に長さ 2〜30 のストロークをランダムに配置したもので，原点 $(0, 0)$ から出発する．数値は空走距離，括弧内は最近傍法からの削減率，最後が処理時間である．

表 2: 実測結果（ランダム順の空走は $N = 2000$ で約 318 000 であり，最近傍法だけで 97 % 減る）．† は 2 秒の上限に達して打ち切った値

N	素朴な最近傍 $O(N^2)$	グリッド最近傍	1. 2-opt + Or-opt（収束まで）	1. 2-opt + Or-opt（100 ms で打切）
500	4708 / 11 ms	4708 / 4 ms	3569 (−24.2 %) / 72 ms	3569 (−24.2 %) / 63 ms
2 000	9160 / 35 ms	9160 / 14 ms	6945 (−24.2 %) / 158 ms	6965 (−24.0 %) / 110 ms
10 000	20026 / 961 ms	20026 / 12 ms	15274 (−23.7 %) / 739 ms	16755 (−16.3 %) / 129 ms
50 000	（未計測）	44257 / 48 ms	35516 (−19.8 %) / 2195 ms†	43670 (−1.3 %) / 281 ms

ここから次のことがわかる．

- グリッド化した最近傍法は $N = 50000$ でも 50 ms 以下で終わり，どの端末でもまず使える．素朴な実装は $N = 10000$ で約 1 秒かかり，ここが現状の改善ポイントである．
- 2-opt + Or-opt は最近傍法からさらに 約 **24 %** 空走を短くする．表 1 の数値と合わせると，最適解から数 % 以内に入っていると推定できる．
- $N \geq 10000$ では 100 ms の打ち切りでは改善しきれない．これは主に区間反転と Or-opt の配列操作のコストで，前節の分割を入れると解消できる見込みである．

なお計測はクラウド上の PC 相当の環境であり，スマートフォンでは 2〜5 倍程度遅くなると見積もっておくのが安全である．

5 推奨する構成

以上より，PlotSender では次の構成を推奨する．

1. 前処理：座標を mm 単位に統一し，端点が一致するストロークを結合する．
2. グループ分け：`structure / fill / detail / delicate` ごと（大きければさらにタイルごと）に分割する．
3. 構築：グリッド版最近傍法で初期解を作り，すぐにプレビューを表示する．
4. 改善：Web Worker の中で 2-opt と Or-opt を交互に，時間予算（例：100〜300 ms，端末が速ければ延長）まで回す．改善のたびに途中結果を画面に送ると，ユーザーには「だんだん良くなる」ように見える．
5. 保険：N が極端に大きいときは最近傍法の代わりにヒルベルト曲線順を使う．

6 コーディング上の注意

6.1 型付き配列（TypedArray）を使う

最も効くのはデータの持ち方である．`{x, y}` のオブジェクトを大量に作ると，メモリ確保とガベージコレクションが頻発し，端末によっては処理が止まって見える．座標は種類ごとに型付き配列へまとめる（構造体の配列ではなく配列の構造体）[7]．

表 3: 推奨するデータの型

データ	型	理由
端点座標 `px`, `py`	`Float64Array`	計算は倍精度で行う．`Float32Array` は有効数字が約 7 桁で，距離の差を比べるときに誤差で判定がぶれる
順番 `order`，逆引き `pos`	`Int32Array`	添字は整数．`pos[v]` で「ストローク v は今何番目か」を $O(1)$ で引く
向き `flip`，描画済み `used`	`Uint8Array`	0/1 のフラグ．`flip[v] ^= 1` で反転
近傍リスト `nb`	`Int32Array(2N × K)`	2 次元配列にせず 1 次元に詰める．`nb[p*K + t]`
グリッド	`Int32Array`（CSR 形式）	セルごとの配列を作らず，開始位置の配列と中身の配列の 2 本で表す

端点には「ストローク v の始点 $= 2v$，終点 $= 2v+1$」という整数 ID を振ると，`p >> 1` でストローク番号，`p ^ 1` で反対側の端点が求まり，分岐が減って速い．

6.2 TypeScript で書く場合の型

外部とのやり取り（UI や JSON）には意味のわかる型を，内部の計算には型付き配列を使い，入口で一度だけ変換する．

```
// 外部向け（読みやすさ優先）
interface Point { x: number; y: number }          // 単位は mm に統一
interface Stroke { id: string; start: Point; end: Point; group: StrokeGroup }
type StrokeGroup = "structure" | "fill" | "detail" | "delicate";
interface PlotOrder { order: number[]; reverse: boolean[]; travelMm: number }

// 内部向け（速さ優先）
interface Endpoints { sx: Float64Array; sy: Float64Array; ex: Float64Array; ey:
Float64Array }
interface Tour { order: Int32Array; flip: Uint8Array; pos: Int32Array }
```

6.3 計算の細かい注意

- 比較には距離の **2** 乗を使う．最近傍探索では `Math.sqrt` を省き $dx^2 + dy^2$ で比べる．ただし式 3 のように距離を足し引きする場合は 2 乗のままではいけない（`sqrt` が必要）．
- **`Math.hypot`** は遅い．`Math.sqrt(dx*dx + dy*dy)` を使う．
- 改善判定には閾値を設ける．`gain > 1e-9` のようにしないと，浮動小数点の丸め誤差で「改善したつもりで元に戻る」無限ループが起こりうる．
- ループ内で **`splice`**，**`indexOf`**，**`filter`**，スプレッド構文を使わない．どれも $O(N)$ で，気づかないうちに $O(N^2)$ になる．
- 距離関数を **1** か所にまとめる．X・Y 軸を別々のモーターで同時に動かす機構なら，移動時間は $\max(|dx|, |dy|)$（チェビシェフ距離）に比例するので，そちらの方が実態に合う．パンタグラフのような非線形な機構でも，距離関数を差し替えるだけで済むようにしておく．ただし 2-opt は $d(a,b) = d(b,a)$（対称）を前提としているので，非対称な距離は使えない．

- 乱数は種（シード）付きにする．焼きなまし等を足すときに，同じ入力で同じ結果が出ないとデバッグできない．

6.4 Web Worker で動かす

最適化をメインスレッドで回すと，その間画面が固まる．計算は Web Worker に移し [8]，型付き配列の `buffer` を **Transferable** として渡す（コピーされず一瞬で渡る）．Worker 内では

- `performance.now()` で時間予算を管理する（毎回呼ぶと遅いので 256 回に 1 回程度），
- 改善が進むたびに途中結果を `postMessage` で返す，
- 新しい画像が来たら古い計算を打ち切る（`worker.terminate()` か中断フラグ），

ようにする．時間予算で打ち切れる「いつでも答えを返せる」設計（anytime アルゴリズム）にしておくことが，端末性能の差を吸収する鍵である．

6.5 テスト

- 結果の `order` が $0 \sim N-1$ をちょうど 1 回ずつ含むか（順列になっているか）を毎回確認する．
- 式 2 で空走距離を最初から計算し直した値と，ゲインを積み上げた値が一致するかを確認する．2-opt の実装ミスはほぼこれで見つかる．
- 小さな N（8 本程度）では全探索で最適解を求め，ヒューリスティックの結果と比べる．

7 まとめ

ペンプロッターの描画順は「入口と出口が異なる開路の TSP」であり，最適解は不要だが高速さが必須である．そのため グリッド版最近傍法で初期解を作り，近傍リスト付き **2-opt** と **Or-opt** を時間予算の範囲で回す 構成が最もバランスが良い．2-opt では区間の反転と同時に各ストロークの向きも反転することで，ゲインを 4 回の距離計算で評価できる．実装面では型付き配列，Web Worker，時間予算による打ち切りが速さの要となる．

8 付録：参考実装

本レポートの計測に使った実装を `order.js` として同じフォルダに置いた．主な関数は次のとおりである．

- `nearestNeighbor(sx, sy, ex, ey, homeX, homeY)`：グリッド版最近傍法．`{order, flip}` を返す．
- `improve(sx, sy, ex, ey, init, {timeLimitMs, K, orOpt})`：2-opt + Or-opt による改善．
- `travel(sx, sy, ex, ey, result)`：空走距離の計算．

2-opt の中心部分は次のように短く書ける．

```
// 位置 i..j を逆順にし，各ストロークの向きも反転する
function reverse(i, j) {
  while (i < j) {
    const a = order[i], b = order[j];
    order[i] = b; order[j] = a; pos[b] = i; pos[a] = j;
    flip[a] ^= 1; flip[b] ^= 1; i++; j--;
  }
  if (i === j) flip[order[i]] ^= 1;
```

```
}
// ゲイン（式の Δ）。ent(k)/ext(k) は位置 k の入口/出口の端点
const gain = (i, j) =>
  dd(ext(i-1), ent(i)) + dd(ext(j), ent(j+1))
  - dd(ext(i-1), ext(j)) - dd(ent(i), ent(j+1));
```

参考文献

[1] David S. Johnson， Lyle A. McGeoch， The Traveling Salesman Problem: A Case Study in Local Optimization， Local Search in Combinatorial Optimization， John Wiley, Sons， p.215–310， 1997

[2] Jon Louis Bentley， Fast Algorithms for Geometric Traveling Salesman Problems， ORSA Journal on Computing， p.387–411， 1992

[3] Loren K. Platzman， John J. Bartholdi， Spacefilling Curves and the Planar Travelling Salesman Problem， Journal of the ACM， p.719–737， 1989

[4] G. A. Croes， A Method for Solving Traveling-Salesman Problems， Operations Research， p.791–812， 1958

[5] Ilhan Or，Traveling Salesman-Type Combinatorial Problems and Their Relation to the Logistics of Regional Blood Banking， 1976

[6] Keld Helsgaun，An Effective Implementation of the Lin-Kernighan Traveling Salesman Heuristic，European Journal of Operational Research， p.106–130， 2000

[7] MDN Web Docs，JavaScript の型付き配列，https://developer.mozilla.org/ja/docs/Web/JavaScript/Guide/Typed_arrays， 閲覧日: 2026-10-25

[8] MDN Web Docs， Web Workers API， https://developer.mozilla.org/ja/docs/Web/API/Web_Workers_API， 閲覧日: 2026-10-25